在 SkyStore 类成本放置之上引入副本–纠删混合冗余：基于混合整数线性规划的多区域布局决策

Hybrid Replica–Erasure Redundancy atop SkyStore-Style Cost Placement: Multi-Region Layout via MILP

作者姓名
单位院系
城市，国家
email@example.edu

摘要—**SkyStore** [1] 以货币成本为核心优化目标：在云厂商价目表（区域存储单价 S、跨区域 **egress** 单价 N）下，通过 **write-local**、**replicate-on-read** 与自适应 **TTL** 直方图，在“为副本付存储费”与“读 **miss** 时再付传输费”之间做驱逐决策；盈亏平衡时间 $T_{\text{even}}=N/S$ 刻画了该权衡的结构。然而 **SkyStore** 在冗余形态上始终按整对象体积 V 计费——远端每多一份缓存，存储项均为 $V{\cdot}S{\cdot}T$，未利用纠删码（**Erasure Coding, EC**）在存储账单上「按 V/k 而非 V 计占用」的价格优势。

EC 在工业界被大规模采用的首要动机正是降低存储放大因子：以 (k,m) 编码在保持可恢复性的同时，将持久占用从 r 倍整对象降为约 $(k{+}m)/k$ 倍 [6], [8]；**Uluyol** 等亦从延迟–成本角度论证跨数据中心场景下 **EC** 相对纯复制可逼近更低成本前沿 [10]。**SkyStore** 系策略优化了保留多久（**TTL**），却未在同一价目表下优化保留多大体积（整副本 **vs.** 分片）。

本文在 **SkyStore** 类仿真栈上，以 **TevictRanges**（`PolicyTevictRangesNew`）为 **replica-only** 基线，提出 `PolicyMilpECnewV3`：**TTL** 与驱逐语义不变，仅在写路径直方图桶滚动时用 **MILP** 比较两种计价分支——基地外再存一整副本（存储 $\propto V$）与放置两条外传 **EC** 数据分片（存储 $\propto 2V/k$）——并计入放置阶段的跨区域 **egress**；**RS** 编解码仅消耗计算资源，不产生存储费或运费，实验中与 **SkyStore** 略去操作费类似，不予计价。**IBM** 类轨迹仿真表明，相对 **TevictRanges** 可在存储分项上取得可解释节省，总成本随对象大小、**TTL** 窗口与区域单价差异而变化。

***Index Terms*—云存储；SkyStore；纠删码；多副本；混合冗余；混合整数线性规划；TTL；多区域放置**

I. 引言

现代应用日益跨多个可用区、地域乃至多云部署对象存储，以降低成本、规避厂商锁定并提升容灾能力 [1]。然而，主流商用对象存储（如 Amazon S3、Google Cloud Storage、Azure Blob）通常按**单云内区域**运营；用户若要在远端保留数据以缩短读延迟或满足合规，必须在**存储占用费**与**跨区域网络出站（egress）费**之间做持续权衡 [1], [3]。存储一份完整副本可避免重复跨区拉取，但随区域数与对象体量放大存储账单；不在访问热点区域保留数据则每次读 miss 都需支付可观的传输费用。因此，多区域对象存储本质上是一个随访问模式**动态变化**的**成本管理**问题，而非传统意义上受容量约束的性能型缓存问题 [1]。

A. SkyStore 基线：成本最优的 *TTL* 驱逐与整副本复制

SkyStore [1] 是近期针对上述问题的一条代表性学术路线。它在现有各云 S3 兼容 API 之上提供统一虚拟桶/对象抽象，核心放置策略可概括为两点：**（i）何时复制**——写请求 write-local；读 miss 时按 replicate-on-read 从费用最低的已有副本拉取并在本地建立**整对象副本**；**（ii）何时驱逐**——为每个副本赋予 TTL，通过访问间隔直方图估计 ExpectedCost(TTL)，在存储与潜在

再传输之间选择成本更低的驻留时长。在两区域 base–cache 模型中，盈亏平衡时间 $T_{\text{even}} = N/S$（N 为跨区 egress 单价，S 为存储单价）给出无先验信息时的保守 TTL；SkyStore 在此基础上用**加权直方图**（hist/last）自适应学习工作负载，并在多区域情形将问题分解为**有向区域图上的配对边 TTL** [1]。

仿真与真实多云原型评估表明，SkyStore 相对 Always Store/Evict、固定 T_{even}、TTL-CC [2]、EWMA、SPANStore [3] 及 AWS Multi-Region 等工业基线，在 SNIA IBM 对象存储轨迹（T15/T29/T65/T78/T79）上可取得最高约 6× 量级的成本节省 [1], [4]。需要强调的是：SkyStore 的“复制”在实现与模型中均为**完整对象拷贝**；Fixed Base（FB）模式下基地副本永不驱逐，非基地区域上的额外冗余同样以整副本形式存在。论文在 Related Work 中讨论了 SPANStore 等**成本感知**的地理复制 [3]，以及 Volley 等侧重延迟的放置系统 [5]，但**并未**在 SkyStore 的 TTL–直方图框架内引入纠删码作为可切换的冗余形态。

本仓库仿真器中的 `PolicyTevictRangesNew`（TevictRanges）实现了与 SkyStore 一致的**配对边直方图、可变粒度时间桶与 TTL 刷新语义**，是本文实验的**直接基线**与 MILP 系数的来源；下文若无特别说明，“TevictRanges 基线”即指该策略下的 replica-only 行为。

B. 纠删码在价格上的优势与 SkyStore 的空白

与 SkyStore 一样，本文讨论 EC 时**只从云厂商账单中的两类费用出发：存储占用**（S_r）与**跨区域数据搬运的 egress**（N_{ij}）[1]。**纠删码的编码与解码属于计算开销**（CPU/内存），**既不按 GB· 时间计存储费，也不按 GB 计 egress**；本文仿真与 MILP 目标**均不包含**编解码费用，与 SkyStore 将 PUT/GET 操作费视为相对可忽略项的处理方式一致。整副本在区域 r 驻留 T 时间的存储费约为 $V{\cdot}S_r{\cdot}T$；若改为 (k,m) EC 并在 r 仅保留一片数据分片，则同窗口存储费约为 $(V/k){\cdot}S_r{\cdot}T$。当 k=2 时，**同等驻留窗口下存储分项可近似减半**——这正是 Azure 等系统将 EC 作为默认冗余形态的经济动因 [6]。Plank 等对 EC 的综述亦将**存储效率**（storage efficiency / 放大因子）列为相对多副本的首要收益 [8]；Uluyol 等在地理分布式存储中进一步表明，允许 EC 而不仅是复制，可在**延迟约束下逼近更低的货币成本前沿** [10]。

SkyStore 已在**时间维度**上做了精细的成本优化：通过直方图估计 ExpectedCost(TTL)，决定副本**保留多久**；T_{even} 则给出“存满 T_{even} 的存储费是否值得再付一次 N 拉取”的盈亏点。但在**体积维度**上，SkyStore 隐含假设远端冗余**永远是体积 V 的整对象**：读 miss 后 replicate-on-read 创建的缓存，存储项始终按 V 计入账单。对于**对象较大、TevictRanges 给出较长 TTL 窗口**的情形，整副本的存储项 $V{\cdot}S{\cdot}T$ 往往主导额外冗余的**可计价成本**；此时 EC 数据分片（本文 FB 下 2+1 RS 的两条外传数据片，每片存储按 V/k 计）在**同一 TTL 与价目表**下可显著压低存储分项。MILP 在布局快照阶段比较的分支成本因此仅含**存储占用 + 放置时的 outbound egress**（将整对象或各 data shard 从 `issueRegion` 传到落区），**不含**编解码计算费。

SPANStore [3] 同样以货币成本为目标，但采用写时主动复制且**不建模 TTL 驱逐**，冗余形态也仅为整副本。**尚未有工作**在 SkyStore/TevictRanges 已确定的 TTL 与 ExpectedCost 框架内，对基地外额外冗余比较两条**计价曲线**：整副本分支 $\text{Cost}_{\text{rep}} \approx V{\cdot}S_r{\cdot}T + N_{ij}{\cdot}V$；EC 分支 $\text{Cost}_{\text{ec}} \approx 2{\cdot}(V/k){\cdot}S_r{\cdot}T + \sum_k N_{ij_k}{\cdot}(V/k)$（本文 FB 叙事下基地整对象常数项对两分支相同）。这正是本文 MILP 所要填补的**成本维度空白**。

C. 研究动机与问题陈述

TevictRanges 基线下，跨区读 miss 后在本地创建**体积为 V 的整副本**，驻留窗口 T 来自配对边直方图。从**价目表**看，该决策的边际存储成本约为 $V{\cdot}S_{\text{dst}}{\cdot}T$；若改为在 dst 侧放置 EC 数据片，边际存储可降为 $(V/k){\cdot}S_{\text{dst}}{\cdot}T$（两片则约 $2V/k$）。**EC 在存储价格上占优**的典型情形是：T 较长、V 较大、且 S_r 相对 N_{ij} 较高——即在 SkyStore 的 $T_{\text{even}} = N/S$ 逻辑下，“存储贵”时不仅应缩短 TTL，还可在**体积维度**用 V/k 替代 V 以降低存储分项。**整副本分支**仍可能在 MILP 中胜出，例如：放置 egress 与区域单价组合下两分支总成本接近、需满足 FB 落区约束、或消融实验强制 replica-only；这与编解码无关——编解码**未进入**本文价目表。

该比较依赖**离散形态**、各区域 S_r、边 N_{ij} 以及 TevictRanges 给出的 T 快照，且不同 `issueRegion` 对

应不同系数，难以用手工规则在所有轨迹上稳定选出最低账单，故引入 MILP 在快照时刻求最小线性成本。

本文**不**重新设计 SkyStore 系统，也**不**将驱逐时刻纳入 MILP 的 0–1 变量（否则规模与执行语义将与 `SimulatorV2` 中 "TTL 到期删除物理副本" 的路径脱节）。研究问题精确表述为：**在 TevictRanges 已提供的 TTL/直方图系数与 SkyStore 类计价拓扑下，于离散规划时刻为每个 issueRegion 选择冗余形态 z 与落区 $(x_r, y_{r,k})$，使存储费与放置 egress 的线性化之和最小（不含编解码计算费），且驱逐与 TTL 刷新仍由仿真器执行。**

D. 本文方案与主要贡献

我们在 `SimulatorV2/V3` 上实现 `PolicyMilpECnewV3`，在 `PolicyTevictRangesNew` 之上增加 MILP 层：

- **系数来源**：非基地副本与 EC 数据分片的存储时长来自配对边 TevictRanges 直方图最后一桶 TTL；FB 模式下基地整副本进入目标常数项 f_b（parity 不单独作为 MILP 二元变量）。
- **决策变量**：$z \in \{0, 1\}$ 选择 replica 分支（基地外至多一整副本）或 EC 分支（恰好两条外传数据分片且两槽不同区）；由 `GurobiEc21MilpBuilder` 求解。
- **触发协议**：仅在**写路径**、配对 TTL 直方图跨越 12h 桶且通过 warm 门控时调用 `syncMilpIfHistogramTtlAdvanced` → `runMilpForIssueRegion`；布局按 `issueRegion` 缓存；读路径 TTL 刷新**不**触发 MILP。histogram 未 warm 时退化为 TevictRanges 行为。

主要贡献如下：

- **第一，**在 SkyStore 类仿真与 TevictRanges TTL 语义之上，形式化**副本–EC 混合冗余**的 MILP：与 `buildCoefficients/GurobiEc21MilpBuilder` 一致的 FB/FP 约束分支、2+1 存储叙事及 replica-only/EC-only 消融开关。
- **第二，**给出**写路径触发**与 `decisionCache` 协议，明确 MILP 仅优化布局快照、驱逐由仿真器负责的分责设计，便于复现与代码–公式对照。
- **第三，**在 SNIA IBM 类轨迹（typeA）及 3/6/9 区域配置下，以 TevictRanges 为基线报告总成本及**存储、传输**分项，并明确实验**不计价** RS 编解码的计算开销。

E. 论文结构

第 II 节综述 SkyStore/SPANStore 等**成本导向**放置、EC 的**存储计价优势**及 TTL 缓存相关工作，并定位本文差异。第 III 节给出系统模型、决策变量与问题边界。第 IV 节描述 MILP 构建、Gurobi 求解与实现要点。第 V 节介绍数据集、指标与相对 TevictRanges 的仿真结果。第 VI 节总结并讨论未来工作。

II. 相关工作

（待撰写：成本导向放置 *SkyStore/SPANStore*；*EC* 存储放大因子与价目表 *[6], [8], [10]*；*TTL* 缓存 *[2], [11]*；与本文差别。）

III. 问题陈述

（待撰写：*FB/FP* 模式、*SkyStore* 计价模型、变量 z, x, y、目标、边界。）

IV. 系统设计

（待撰写：*Fig. 1* 流程、*MILP* 公式、算法、实现。）

V. 实验评估

（待撰写：*T15–T79/typeA*、*3/6/9* 区域、相对 *TevictRanges* 基线。）

VI. 结论

（待撰写。）

参考文献

[1] S. Liu, X. Mo, M. Hershcovitch, H. Zhang, A. Cheng, G. Girmonsky, G. Vernik, M. Factor, T. Bang, S. Ponnapalli, N. Crooks, J. E. Gonzalez, D. Harnik, and I. Stoica, "SkyStore: Cost-optimized object storage across regions and clouds," *Proceedings of the VLDB Endowment*, vol. 18, no. 1, pp. 1–14, 2025, arXiv:2502.20818.

[2] D. Carra, G. Neglia, and P. Michiardi, "TTL-based cloud caches," in *IEEE INFOCOM*, 2019, pp. 685–693.

[3] Z. Wu, M. Butkiewicz, D. Perkins, E. Katz-Bassett, and H. V. Madhyastha, "SPANStore: Cost-effective geo-replicated storage spanning multiple cloud services," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 292–308.

[4] O. Eytan, D. Harnik, E. Ofer, R. Friedman, and R. Kat, "IBM object store traces (SNIA IOTTA trace set 36305)," in *SNIA IOTTA Trace Repository*, 2019.

[5] S. Agarwal, J. Dunagan, N. Jain, S. Saroiu, A. Wolman, and H. Bhogan, "Volley: Automated data placement for geo-distributed cloud services," in *Proceedings of the 7th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2010, p. 2.

[6] C. Huang, H. Simitci, Y. Xu, S. Smallberg, B. Calder, J. Li, and A. Patterson, "Erasure coding in windows azure storage," in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2012, pp. 15–26.

[7] K. V. Rashmi, N. B. Shah, and P. V. Kumar, "A solution to the network challenges of data recovery in erasure-coded distributed storage systems: A study on the Facebook warehouse cluster," in *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2014, pp. 575–588.

[8] J. S. Plank, M. Blaum, and J. L. Hafner, "Erasure codes for storage systems: A primer," in *6th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2013.

[9] A. G. Dimakis, P. B. Godfrey, Y. Wu, M. J. Wainwright, and K. Ramchandran, "Network coding for distributed storage systems," *IEEE Transactions on Information Theory*, vol. 56, no. 9, pp. 4539–4551, 2010.

[10] M. Uluyol, A. Huang, A. Goel, M. Chowdhury, and H. V. Madhyastha, "Near-optimal latency versus cost tradeoffs in geo-distributed storage," in *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 157–180.

[11] S. Basu, A. Sundarrajan, J. Ghaderi, S. Shakkottai, and R. Sitaraman, "Adaptive TTL-based caching for content delivery," *IEEE/ACM Transactions on Networking*, vol. 26, no. 3, pp. 1063–1077, 2018.